

IT UNIVERSITY OF COPENHAGEN

Department of Computer Science
KISPECI1SE - Master Thesis

Formalising Commitment Schemes in Nominal-SSProve

Prepared by: Peter Gjeraae Sabroe (pesa@itu.dk)

Supervised by: Carsten Schürmann,
Markus Krabbe Larsen

Date: June 2, 2025

Abstract

Nominal-SSProve, a framework under development at the IT University of Copenhagen for the formal verification of cryptographic protocols, is designed as a simplified refinement of SSProve. As part of this effort, we define a parameterized and modular abstraction of commitment schemes – treating them as first-class components, rather than tightly coupled constructions within Σ -protocols, as in SSProve. We show how such schemes can be instantiated from Σ -protocols and formally prove their security using the state-separating proof methodology. The result is a flexible, reusable component, fully formalized in Coq and well-suited for building and verifying cryptographic constructions.

Contents

1	Introduction	1
1.1	Related Work	1
2	Cryptographic primitives	1
2.1	Commitment schemes	1
2.2	Σ -protocols	2
3	Nominal-SSProve	3
3.1	Syntax and semantics	4
4	Implementing Commitment Schemes	5
4.1	Commitment scheme from Σ -protocol	5
5	Proving security	8
5.1	Correctness	8
5.2	Hiding	11
5.3	Binding	13
6	Commitment scheme from Schnorr	17
7	Conclusion	17

1 Introduction

In recent years, a variety of frameworks for formalising cryptographic protocols have emerged from research institutions worldwide. While these frameworks offer broadly similar tools for mechanization, they differ in implementation strategies, logical foundations and semantic models – each bringing its own advantages and limitations. A new framework, Nominal-SSProve (see section 3), is currently being developed at the IT University of Copenhagen. The work presented in this paper can be seen as a contribution to this ongoing research effort.

A common feature of these frameworks is that they all provide formal definitions and mechanizations of standard cryptographic primitives, such as public key encryption schemes, Σ -protocols, message authentication codes (MACs), and more. We provide a formalisation of another such primitive within the Nominal-SSProve framework – namely, commitment schemes. Our implementation offers a clear, parameterized formulation, that enables users to construct commitment schemes from scratch. Furthermore, it allows for the straightforward transformation of already proven Σ -protocols into commitment schemes without the need for additional effort.

1.1 Related Work

In the SSProve framework [10] – a cryptographic formalization environment for `Coq` – Haselwarter et al. demonstrate how to construct a commitment scheme from a Σ -protocol. While their proof strategy, which involves game-hopping and reasoning about the advantages of composed packages, bears some resemblance to ours, the overall approach differs significantly. In our work, commitment schemes are defined independently, allowing users to specify a commitment scheme *without* necessarily linking it to a Σ -protocol. In contrast, SSProve tightly couples the two primitives. Furthermore, our proof of the hiding property more directly reduces to the SHVZK property of the underlying Σ -protocol. Our binding proof also highlights the connection between binding and the hardness of the relation R on which the Σ -protocol is based.

Butler et al. [4] demonstrate how to construct the two primitives independently and explore their relationship using the CryptHOL framework. Their approach is more closely aligned with the parameterized and abstract strategy we adopt. However, our proof strategy differs in that it leverages a hybrid, game-hopping approach, where the adversary interacts with the games externally. In contrast, Butler et al. employ a more traditional game-playing style, where the adversary is passed as input between games.

The Pedersen commitment scheme has been formalised in EasyCrypt by Metere and Dong [12], in a style similar to the formalisation in CryptHOL by Butler.

2 Cryptographic primitives

2.1 Commitment schemes

A Commitment scheme, as introduced by Blum [2] and Even [9], is a protocol between a Prover, P , and a Verifier, V . P publicly commits to a *value*, by computing and sharing a *commitment* based on the *value*. Later in time P

reveals the *value* and V can check whether the *value* and *commitment* correspond. The exact definition of commitment schemes vary in the literature. In following we use the definition from Butler et al. [4].

Definition 1. A commitment scheme has the following three-part form:

1. **Key generation:** $k \leftarrow \text{setup}()$. Outputs a key k used by the protocol.
2. **Commitment phase:** $(c, o) \leftarrow \text{commit}(k, v)$. Generates a commitment c and an opening o , based on the key k and the value v committed to.
3. **Verification phase:** $b \leftarrow \text{verify}(k, c, v, o)$. The verify algorithm takes as input the key k , commitment c , value v , and opening o , and outputs a boolean indicating if the verification was successful.

Among the alternative definitions are Dodis [8] substituting `verify()` with `open()`, outputting a *value* instead of a boolean, and Schoenmakers [14] leaving out the `verify()` algorithm all together and instead using the `commit()` algorithm in the verification phase.

A commitment scheme must satisfy three properties:

1. **Correctness:** If both parties follow the protocol honestly, `verify()` will return `true`.
2. **Hiding:** A commitment scheme must be *hiding*, meaning that the distributions resulting from `commit( $k, v_1$ )` and `commit( $k, v_2$ )` are indistinguishable – i.e., the *commitments* reveal no information about the *values* v_1 and v_2 .
3. **Binding:** A commitment scheme must be *binding*, meaning that it is infeasible to compute the same *commitment* c for two different *values* v_1 and v_2 .

Furthermore a distinction is made as to whether the *binding* and *hiding* properties are computationally bounded, i.e. we limit the adversary to be a p.p.t. algorithm, or perfect. Schoenmakers [14] gives an informal argument showing, that if a commitment scheme C has perfect hiding, it can only have computational binding and vice versa.

2.2 Σ -protocols

A Σ -protocol is a two-party cryptographic protocol in which a Prover, P , convinces a Verifier, V , of possession of some knowledge without disclosing it. The communication alternates back and forth between the parties, illustrated by the Greek letter ' Σ ' — hence the name. In short, P first computes a *message*, a , and sends it to V . V responds by sending a *challenge*, e , to P . Based on e , P computes a *response*, z , and sends it to V , and V then verifies whether (a, e, z) is an accepting conversation.

The protocol is based on a relation R , where if $(h, w) \in R$, h is the *statement* (public input) and w is the corresponding *witness* (private input to P - the knowledge that, P wants to convince V he possesses). Σ -protocols are widely

used in identification schemes, Zero-Knowledge Proofs, among others, and it is regarded a fundamental cryptographic tool. Multiple definitions exist in the literature [7] [5] [14], in the present text we again refer to the definition given by Butler et al. [4], replacing Honest Verifier Zero Knowledge with *Special* Honest Verifier Zero Knowledge (SHVZK):

Definition 2. Assume a protocol, π , of the above form run between P and V . Then π is a Σ -protocol for a relation R if the following properties hold:

1. **Completeness:** if P and V follow the protocol on public input h and private input w such that $(h, w) \in R$, then V always accepts.
2. **Special soundness:** there exists an adversary, A , such that when given a pair of accepting conversations (a, e, z) and (a, e', z') where $e \neq e'$ it can compute w such that $(h, w) \in R$
3. **Special Honest verifier Zero-Knowledge (SHVZK)** There exists a polynomial-time simulator S that on input h and e outputs an accepting conversation (a, e, z) with the same probability distribution as the real conversations between P and V on input (h, w) .

HVZK is a somewhat weaker property than SHVZK, in which the simulator, S , samples e internally, instead of taking it as an input. Barthe et al. [1] have shown, that SHVZK implies HVZK.

3 Nominal-SSProve

State-separating proofs as presented by Brzuska et al. [3] have become a widely accepted method for reasoning about security of cryptographic protocols. The core idea is to model games, adversaries, and reductions as stateful packages that can be composed. This enables the use of *game-hopping* – a powerful technique in which the security of a complex game is reduced to the security of a sequence of intermediate games that are easier to analyze. This approach is also referred to as the *hybrid* technique by Rosulek [13]. We will exploit this technique repeatedly in section 5 when proving the security of commitment schemes. Furthermore, in Nominal-SSProve we always model security in terms of the *advantage* an adversary has in distinguishing between two games; we elaborate on this in section 5

The main issue with state-separating proofs is that the local state (where a package may keep safety-critical information, e.g. a private key) must be kept hidden from the adversary. Brzuska et al. [3] refer to tacit renaming of state variables to keep information safe, without specifying how this renaming should be carried out. In SSProve [10] they circumvent the issue by explicitly assuming disjointness between packages. This results in substantially larger proofs, because the disjointness assumptions must be accounted for.

Nominal-SSProve [11] provides an elegant solution to this issue, extending state-separating proofs with nominal sets. In this way each package has its own local state space, making state variable capture impossible. Furthermore state variables are automatically renamed when packages are composed, resulting in

a semantically transparent framework, where the user need not worry about state capture.

3.1 Syntax and semantics

To facilitate a better understanding of the games presented later, we provide an overview of some of the most common syntax used in Nominal-SSProve, along with its corresponding semantic.

Table 1: Overview of Nominal-SSProve syntax and semantics

Syntax / Concept	Semantic / Translation
Operation, <code>opsig</code>	An operation is a function associated with an identifier, a source type, and a target type — the latter two together forming the operation’s signature. As an example, <code>#[INIT] : 'unit → 'unit</code> is an operation with identifier <code>INIT</code>, source type <code>unit</code>, and target type <code>unit</code>. Under the hood, identifiers are natural numbers; names are for readability.
<code>code</code>	We use <code>code</code> to implement operations. It is written between curly braces with instructions separated by <code>;;</code>. Technically, it is implemented as a free monad (<code>raw_code</code>) to enable the modelling of side effects within Gallina, the purely functional language on which <code>Coq</code> is built.
<code>←</code>	Used to bind the result of a computation to a variable inside <code>code</code>. For example, <code>'k ← p.(setup)</code> binds the computation of <code>setup</code> to the variable <code>k</code>.
Location	We store information generated in our games in a <code>Location</code>, such as public keys, secret keys, flags, and other relevant data. A location is defined by a content type and an identifier (natural number): <code>Definition Location := (t : choice_type), nat.</code> The <code>raw_code</code> free monad provides <code>get</code> and <code>put</code> to read/write locations.
Interface	An <code>interface</code> is a finite set of operations (<code>opsig</code>). Interface variables are typically prefixed with <code>I</code>, e.g., <code>IBinding</code>, <code>ICommitment</code>.
Module	A <code>module</code> is the primary structure used to model cryptographic games. It has an import <code>interface</code> (for external calls), an export <code>interface</code> (for provided operations), and a set of locations for reads/writes. Module implementations must define the operations promised in the export interface via <code>code</code>.
$M_1 \circ M_2$	Two <code>modules</code>, M_1 and M_2, can be sequentially composed using the <code>◦</code> syntax if the import interface of M_1 matches the export interface of M_2.

Syntax / Concept	Semantic / Translation
<code>game</code>	A predefined <code>module</code> with an empty <code>import interface</code> , hence only requiring an <code>export interface</code> . By definition <code>games</code> has no imports [10]
<code>adversary</code>	A predefined <code>module</code> with a special <code>export interface</code> , <code>A_export</code> , hence only requiring an <code>import interface</code> . <code>A_export</code> only has one operation <code>RUN</code> with signature <code>'unit → 'bool</code> .
<code>Adv</code>	A function taking as input two <code>games</code> and an <code>adversary</code> , returning the quantified advantage by which the <code>adversary</code> can distinguish between the two games. The <code>export interface</code> of the two games must be identical (however, their implementation may differ), and it must match the <code>import interface</code> of the <code>adversary</code> .
<code>AdvFor</code>	Same as <code>Adv</code> , but taking a <code>game-pair</code> (a boolean function, choosing between two games) instead of two games as input.
<code>perfect</code>	A function taking an <code>import interface</code> (of the <code>adversary</code>) and two <code>games</code> G_1, G_2 as input and wraps them in a <code>AdvFor</code> statement, claiming the quantified advantage of any distinguishing <code>adversary</code> is 0, i.e. the games are <i>perfectly indistinguishable</i> ; also written $G_1 \approx_0 G_2$

4 Implementing Commitment Schemes

Inspired by the implementation of Σ -protocols in Nominal-SSProve (see fig. 2), we define the commitment scheme abstractly as the record `raw_com` depicted in fig. 1. Keys, values, commitments and openings are defined as separate types promising to be a `choice_type` – a specialised type in SSProve allowing for sampling, `code`-use, `Location`-use, and more. The three algorithms of our commitment scheme as described in definition 1 are part of the record, implemented as `code` – specifying the input and output, as well as no `Locations` (`no_locs`) and empty `interfaces`. Lastly we define the function `sample_value` to enable sampling of `Values` in the abstract context, since sampling in SSProve is only defined for *inhabitant* types. Our types (`Value`, `Commitment`, etc.) in the `raw_com`-record only becomes inhabitant once the record is instantiated. The complete implementation can be found in the `Com.v` file¹.

4.1 Commitment scheme from Σ -protocol

As shown by Damgård [6] a *perfectly hiding* commitment scheme with *computational binding* can be constructed from a Σ -protocol. Our goal is to define a function that transforms a `raw_sigma` from fig. 2 into a `raw_com`. To facilitate

¹<https://github.com/petersabroe/thesis/blob/main/project/theories/Com.v>


```

Record raw_com :=
{ Key : choice_type
; Value : choice_type
; Commitment : choice_type
; Opening : choice_type

; setup :
  code no_locs [interface] Key

; commit :
  ∀ (k : Key) (v : Value),
    code no_locs [interface] (Commitment × Opening)

; verify :
  ∀ (k : Key) (c : Commitment) (v : Value) (d :
    ⇔ Opening),
    code no_locs [interface] bool

; sampl_value :
  code no_locs [interface] Value
}.
    
```

Figure 1: Defining commitment scheme record

```

Record raw_sigma :=
{ Statement : choice_type
; Witness : choice_type
; Message : choice_type
; State : choice_type
; Challenge : choice_type
; Response : choice_type

; R : Statement → Witness → bool

; commit :
  ∀ (h : Statement) (w : Witness),
    code no_locs [interface] (Message × State)

; response :
  ∀ (h : Statement) (w : Witness)
  (a : Message) (s : State) (e : Challenge),
    code no_locs [interface] Response

; verify :
  ∀ (h : Statement) (a : Message) (e : Challenge)
  (z : Response),
    bool

; simulate :
  ∀ (h : Statement) (e : Challenge),
    code no_locs [interface] (Message × Response)

; extractor :
  ∀ (h : Statement) (a : Message)
  (e : Challenge) (e' : Challenge)
  (z : Response) (z' : Response),
    'option Witness
}.
    
```

 Figure 2: Defining Σ -protocol record

```

Record raw_sigExt :=
{ p :> raw_sigma

; sampl_challenge :
  code no_locs [interface] (Challenge p)

; key_gen :
  code no_locs [interface] (Witness p ×
    ⇔ Statement p)
}.

Definition sig_to_com (p : raw_sigExt) : raw_com :=
{| Key := p.(Statement)
; Value := p.(Challenge)
; Commitment := p.(Message)
; Opening := p.(Response)

; setup :=
  {code
    '(w, h) ← p.(key_gen) ;;
    #assert p.(R) h w ;;
    ret ((h) : _)
  }

; commit := λ k v,
  {code
    '(a, z) ← p.(simulate) k v ;;
    ret ((a, z) : (_ × _))
  }

; verify := λ k c v d,
  {code
    ret (p.(Sigma.verify) k c v d)%B
  }

; sampl_value := p.(sampl_challenge)
|}.
    
```

 Figure 3: Creating a commitment scheme from a Σ -protocol

```

Definition Correct_real p :
  game (ICorrect_com p) :=
  [module no_locs ;
    #def #[ CORRECTNESS ] (v : 'value p) : ('bool)
    {
      k ← p.(setup) ;;
      '(c, o) ← p.(commit) k v ;;
      b ← p.(verify) k c v o ;;
      ret b
    }
  ].

Definition Correct_ideal p :
  game (ICorrect_com p) :=
  [module no_locs ;
    #def #[ CORRECTNESS ] (v : 'value p) : ('bool)
    {
      k ← p.(setup) ;;
      ret true
    }
  ].
    
```

Figure 4: Defining the Correctness-games

```

Definition Hiding_real p :
  game (ICommitment p) :=
  [module no_locs ;
    #def #[ COMMITMENT ] (v : 'value p) :
    ⇨ ('commitment p)
    {
      k ← p.(setup) ;;
      '(c, o) ← p.(commit) k v ;;
      @ret ('commitment p) c
    }
  ].

Definition Hiding_ideal p :
  game (ICommitment p) :=
  [module no_locs ;
    #def #[ COMMITMENT ] (v : 'value p) :
    ⇨ ('commitment p)
    {
      k ← p.(setup) ;;
      u ← p.(sampl_value) ;;
      '(c, o) ← p.(commit) k u ;;
      @ret ('commitment p) c
    }
  ].
    
```

Figure 5: Defining the Hiding-games

```

Definition Binding_real p :
  game (IBinding p) :=
  [module fset [:: key_loc p] ;
    #def #[ INIT ] (_ : 'unit) : ('unit) {
      'k ← p.(setup) ;;
      #put key_loc p := Some k ;;
      ret tt
    } ;

    #def #[ GET ] (_ : 'unit) : ('key p) {
      k ← getSome key_loc p ;;
      ret k
    } ;

    #def #[ BINDING ] ('(c, v, o, v', o') :
    ⇨ 'binding p) : 'bool
    {
      k ← getSome key_loc p ;;
      b ← p.(verify) k c v o ;;
      b' ← p.(verify) k c v' o' ;;
      let b'' := (v != v') in
      @ret 'bool (b && b' && b'')
    }
  ].

Definition Binding_ideal p :
  game (IBinding p) :=
  [module fset [:: key_loc p] ;
    #def #[ INIT ] (_ : 'unit) : ('unit) {
      'k ← p.(setup) ;;
      #put key_loc p := Some k ;;
      ret tt
    } ;

    #def #[ GET ] (_ : 'unit) : ('key p) {
      k ← getSome key_loc p ;;
      ret k
    } ;

    #def #[ BINDING ] ('(c, v, o, v', o') :
    ⇨ 'binding p) : 'bool
    {
      k ← getSome key_loc p ;;
      b ← p.(verify) k c v o ;;
      b' ← p.(verify) k c v' o' ;;
      let b'' := (v != v') in
      @ret 'bool (false && b && b' && b'')
    }
  ].
    
```

Figure 6: Defining the Binding-games

this, we first introduce the intermediate record `raw_sigExt`, shown in fig. 3. This record – which extends the `raw_sigma` with additional sampling and key generation functionality needed for implementing the commitment scheme – can be used anywhere a `raw_sigma` is expected due the *coercion*-syntax `p :> raw_sigma`. The function `sig_to_com` (also fig. 3) takes a `raw_sigExt` and creates a `raw_com` from it, in line with Damgård’s description [7].

5 Proving security

The three fundamental properties of a commitment scheme – namely, *correctness*, *hiding*, and *binding* – are formalized as pairs of games; each pair having a unique export interface. The correctness games in fig. 4 have the export interface `ICorrect_com` with one operation, `CORRECTNESS`; the hiding games in fig. 5 have the export interface `ICommitment` also with one operation, `COMMITMENT`; and the binding games in fig. 6 have the export interface `IBinding` with three operations, `INIT`, `GET` and `BINDING`.

We adopt a real-versus-ideal paradigm, which is standard practice in the context of state-separating proofs. Briefly, the *ideal* game yields either the expected output or a perfectly random value, depending on the property being modelled, whereas the *real* game reflects the outcome of an actual execution. This game-based formulation facilitates rigorous reasoning about the advantage of any adversary attempting to distinguish between the two settings.

All games take a parameter `p` of type `raw_com`, reflecting the parameterized methodology employed in Nominal-SSProve. This design enables us to formulate general, abstract definitions while still allowing access to local types and functions encapsulated within the record. We adopt this approach consistently throughout, parameterizing our modules, functions, reductions, and related components with a `p` – which may be of type `raw_com` or, in some cases, `raw_sigExt`, depending on the context.

In the following subsections, we first define the advantage of the adversary abstractly for each property. We then demonstrate how a commitment scheme instantiated via a Σ -protocol satisfies these properties, thereby yielding a secure commitment scheme.

5.1 Correctness

In the function below, `Adv_Correct`, we define a bound to the advantage of the `Correctness`-games. The function takes two parameters; `p`, of type `raw_com`, and ϵ , a function taking an adversary with import interface `ICorrect_com` and returning a real number, which expresses the quantified advantage. We use this ϵ -function to bound the advantage of our `Correctness`-games, i.e. `AdvFor (Correct p) A`, where `(Correct p)` expresses the game-pair `Correct_real` vs `Correct_ideal`.

```
Definition Adv_Correct p ( $\epsilon$  : adversary (ICorrect_com p)  $\rightarrow$  Axioms.R) :=
   $\forall$  A : adversary (ICorrect_com p), AdvFor (Correct p) A <=  $\epsilon$  A.
```

Proof strategy and lemmas

We want to show that correctness of the commitment scheme is bounded by correctness and the SHVZK property of the Σ -protocol. The following is already

defined and proved in the Nominal-SSProve file `Sigma.v`²:

```
Definition Correct_sim p : raw_module := Verify_call p ∘ SHVZK_ideal p.

Lemma Adv_Correct_sim p (A : adversary (ICorrect p)) :
  Adv (Correct_sim p) (Sigma.Correct_ideal p) A
  <= AdvFor (SHVZK p) (A ∘ Verify_call p) + AdvFor (Sigma.Correct p) A.
```

Note the ambiguity of names, since both `Com.v` and `Sigma.v` define modules `Correct_ideal` and `Correct_real`. For clarification, we refer to the modules in `Sigma.v` by prefixing them `Sigma.Correct_xx`. Our proof strategy is to show the following two perfect indistinguishabilities:

$$\begin{aligned} \text{Sigma.Correct_ideal } p &\approx_0 \text{ Correct_ideal (sig_to_com } p) \\ \text{Correct_sim } p &\approx_0 \text{ Correct_real (sig_to_com } p) \end{aligned}$$

In the above `p` is of type `raw_sigExt`, therefore we must use our `sig_to_com` transformation function (fig. 3) to turn it into the `raw_com`-type that our commitment scheme games expect.

In order to compare the games they must export the same interface. We create a reduction module, `Call_correct_sig` (see fig. 7), which imports the interface exported by the `Correctness`-games in the `Sigma.v` file, and exports the `ICorrect_com`-interface matching the `Correctness`-games in `Com.v`. Now we can formalise two lemmas using the `perfect` syntax of Nominal-SSProve:

```
Lemma Correct_ideal_sim_perf p :
  perfect (ICorrect_com (sig_to_com p))
    (Call_correct_sig p ∘ Sigma.Correct_ideal p) (Correct_ideal (sig_to_com p)).

Lemma Correct_real_sim_perf p :
  perfect (ICorrect_com (sig_to_com p))
    (Call_correct_sig p ∘ Sigma.Correct_sim p) (Correct_real (sig_to_com p)).
```

Proof. For proofs of both lemmas see Coq-code in `Com.v`. □

Theorem

Using `Adv_Correct` defined in the beginning of this section, we formalise a theorem for correctness of the commitment scheme, by instantiating ε with `AdvFor (SHVZK p) + AdvFor (Sigma.Correct p)`.

```
Theorem Com_Correct_Correct:
  ∀ (p : raw_sigExt) ,
  Adv_Correct (sig_to_com p) (λ A,
    AdvFor (SHVZK p) (A ∘ Call_correct_sig p ∘ Verify_call p) +
    AdvFor (Sigma.Correct p) (A ∘ Call_correct_sig p)).
```

Proof. `Adv_Correct` unfolds to `Adv Correct_real Correct_ideal`. We can replace the games with `Correct_sim` and `Sigma.Correct_ideal` respectively, using the lemmas proved above. This makes the theorem identical to the `Adv_Correct_sim`-lemma proved in the `Sigma.v` file shown above. For a formal proof see Coq-code in `Com.v`. □

²<https://github.com/MarkusKL/nominal-ssprove/blob/master/theories/Example/Sigma/Sigma.v>

```

Definition Call_correct_sig (p: raw_sigExt) :
  module (ICorrect p) (ICorrect_com (sig_to_com
    ↪ p)) :=
  [module no_locs ;
    #def #[ CORRECTNESS ] (v : 'value
      ↪ (sig_to_com p)) : 'bool
    {
      #import {sig #[ RUN ] : ('input p) →
        ↪ 'bool} as COR ;;
      'w, h ← p.(key_gen) ;;
      b ← COR (h, w, v) ;;
      ret b
    }
  ].
    
```

Figure 7: Correctness reduction

```

Definition Call_SHVZK_inp (p: raw_sigExt) :
  module (Transcript p) (ICommitment (sig_to_com
    ↪ p)) :=
  [module no_locs ;
    #def #[ COMMITMENT ] (v : 'value
      ↪ (sig_to_com p)) : ('commitment
      ↪ (sig_to_com p))
    {
      #import {sig #[ TRANSCRIPT ] :
        ↪ ('input p) → 'transcript p} as
        ↪ TRANS ;;
      'w, h ← p.(key_gen) ;;
      #assert p.(R) h w ;;
      'h, a, e, z ← TRANS (h, w, v) ;;
      ret (a : (sig_to_com p).(Commitment))
    }
  ].
    
```

Figure 8: Hiding reduction using input

```

Definition Call_SHVZK_sam (p: raw_sigExt) :
  module (Transcript p) (ICommitment (sig_to_com
    ↪ p)) :=
  [module no_locs ;
    #def #[ COMMITMENT ] (v : 'value
      ↪ (sig_to_com p)) : ('commitment
      ↪ (sig_to_com p))
    {
      #import {sig #[ TRANSCRIPT ] :
        ↪ ('input p) → 'transcript p} as
        ↪ TRANS ;;
      'w, h ← p.(key_gen) ;;
      #assert p.(R) h w ;;
      u ← (sig_to_com p).(sampl_value) ;;
      'h, a, e, z ← TRANS (h, w, u) ;;
      ret (a : (sig_to_com p).(Commitment))
    }
  ].
    
```

Figure 9: Hiding reduction using sample

```

Definition Call_Soundness (p: raw_sigExt) :
  module (Soundness p) (IBinding (sig_to_com p)) :=
  [module fset [:: key_loc (sig_to_com p) ] ;
    #def #[ INIT ] (_ : 'unit) : ('unit) {
      'k ← (sig_to_com p).(setup) ;;
      #put key_loc (sig_to_com p) := Some k ;;
      ret tt
    } ;

    #def #[ GET ] (_ : 'unit) : ('key (sig_to_com p)) {
      k ← getSome key_loc (sig_to_com p) ;;
      ret k
    } ;

    #def #[ BINDING ] (('c, v, o, v', o') : 'binding (sig_to_com
      ↪ p)) : 'bool
    {
      h ← getSome (key_loc (sig_to_com p)) ;;
      #import {sig #[ SOUNDNESS ] : ('soundness p) → 'bool} as
        ↪ SOUND ;;
      'b ← SOUND ((h, c), ((v, o), (v', o'))) ;;
      ret b
    }
  ].
    
```

Figure 10: Reduction from Binding to Special Soundness

```

Definition Call_Hardness (p: raw_sigExt) :
  module (IHardness p) (IBinding (sig_to_com p)) :=
  [module no_locs ;
    #def #[ INIT ] (_ : 'unit) : ('unit) {
      #import {sig #[ INIT ] : 'unit → 'unit} as INITH ;;
      u ← INITH Datatypes.tt ;;
      ret u
    } ;

    #def #[ GET ] (_ : 'unit) : ('key (sig_to_com p))
    {
      #import {sig #[ GET ] : 'unit → 'statement p} as GETH ;;
      h ← GETH Datatypes.tt ;;
      ret h
    } ;

    #def #[ BINDING ] (('c, v, o, v', o') : 'binding (sig_to_com
      ↪ p)) : 'bool
    {
      #import {sig #[ QUERY ] : ('witness p) → 'bool} as QUE
        ↪ ;;
      #import {sig #[ GET ] : 'unit → 'statement p} as GETH
        ↪ ;;
      h ← GETH tt ;;
      let b := p.(Sigma.verify) h c v o in
      let b' := p.(Sigma.verify) h c v' o' in
      let b'' := (v != v') in
      let ow := p.(extractor) h c v v' o o' in
      if ow is Some w
        then 'b''' ← QUE w ;; ret (b''' && b && b' && b'')
        else ret false
    }
  ].
    
```

Figure 11: Reduction from Binding to Hardness

5.2 Hiding

The *hiding* property of the commitment scheme is defined as the indistinguishability of the distributions induced by two calls to `commit` [14]. In the context of cryptographic game-playing we define it as the **Hiding**-games in fig. 5. The **Hiding_{real}** game commits to a value, v , provided as input to the game, and returns the commitment, c . The **Hiding_{ideal}** game on the other hand, samples a value, u , internally, commits to it and returns the commitment, c . We abstractly bound the advantage of the **Hiding**-games, **Adv_{Hiding}**, the same way did the **Correctness**-games, using an ε -function:

Definition **Adv_{Hiding}** p (ε : adversary (ICommitment p) $\rightarrow$ Axioms. R) :=
 $\forall A$: adversary (ICommitment p), AdvFor (Hiding p) $A \leq \varepsilon A$.

Proof strategy and lemmas

We want to show that the advantage of the **Hiding**-games is bounded by the advantage of the **SHVZK**-games of the underlying Σ -protocol (see fig. 12).

```
Definition SHVZK_real p :
game (Transcript p) :=
[module no_locs ;
  #def #[ TRANSCRIPT ] ('(h, w, e) : 'input p) : ('transcript p) {
    #assert p.(R) h w ;;
    '(a, s) ← p.(commit) h w ;;
    z ← p.(response) h w a s e ;;
    @ret (chTranscript p) (h, a, e, z)
  }
].
```

```
Definition SHVZK_ideal p :
game (Transcript p) :=
[module no_locs ;
  #def #[ TRANSCRIPT ] ('(h, w, e) : 'input p) : ('transcript p) {
    #assert p.(R) h w ;;
    '(a, z) ← p.(simulate) h e ;;
    @ret (chTranscript p) (h, a, e, z)
  }
].
```

Figure 12: The SHVZK-Games as defined in **Sigma.v** of Nominal-SSProve

Recall that, in our construction of a commitment scheme from a Σ -protocol (see fig. 3), the `commit` function is defined in terms of the protocol's `simulate` function. Consequently, our **Hiding**-games closely resemble the **SHVZK_{ideal}** game, which naturally facilitates the first two perfect indistinguishabilities we aim to establish.

$$\begin{aligned} \text{Hiding_real (sig_to_com } p) &\approx_0 \text{ SHVZK_ideal } p \\ \text{Hiding_ideal (sig_to_com } p) &\approx_0 \text{ SHVZK_ideal } p \end{aligned}$$

In order to compare the games, we need a reduction module to link the hiding adversary to the **SHVZK_{ideal}** game. To this end, we define two reduction modules:

`Call_SHVZK_inp`, which forwards the externally provided value v to the `SHVZK_ideal` game, and `Call_SHVZK_sam`, which instead forwards an internally sampled value u (see fig. 8 and fig. 9, respectively). These reductions capture the differing behaviors of the `Hiding`-games. With these in place, we can formally state the perfect indistinguishability results as the following lemmas:

```
Lemma Hiding_real_SHVZK_ideal_perf p :
  perfect (ICommitment (sig_to_com p))
    (Hiding_real (sig_to_com p)) (Call_SHVZK_inp p ◦ SHVZK_ideal p).
```

```
Lemma Hiding_ideal_SHVZK_ideal_perf p :
  perfect (ICommitment (sig_to_com p))
    (Hiding_ideal (sig_to_com p)) (Call_SHVZK_sam p ◦ SHVZK_ideal p) .
```

Proof. See Coq-code. □

We also want to show that the two reductions themselves are perfectly indistinguishable when linked to the `SHVZK_real` game:

$$\text{Call_SHVZK_inp } p \circ \text{SHVZK_real } p \approx_0 \text{Call_SHVZK_sam } p \circ \text{SHVZK_real } p$$

This is an essential part of the proof, which Damgård also pinpoints [7]. First, observe that the returned element, a , in both reductions is of type `Commitment`, which corresponds to type `Message` in the Σ -protocol defined in fig. 3. This element is produced within the `SHVZK_real` game. Upon inspecting the behavior of `SHVZK_real`, we note that the message, a , is generated independently of the challenge, e . Recall that the type `Challenge` in the Σ -protocol corresponds to type `value`, and thus the specific value (or challenge) forwarded to the `SHVZK_game` from the reduction module has no influence on the resulting message (or commitment). Consequently, it is irrelevant whether the challenge is sampled internally or supplied as input. We can now formally state this perfect indistinguishability as the lemma below (Note: the `NoFail` hypotheses are necessary in Coq to allow either game to perform a computation in isolation, as long as it doesn't fail):

```
Lemma Red_perf (p: raw_sigExt) :
  (forall h w a s e, NoFail (response p h w a s e)) ->
  NoFail (sampl_challenge p) ->
  perfect (ICommitment (sig_to_com p))
    (Call_SHVZK_inp p ◦ SHVZK_real p) (Call_SHVZK_sam p ◦ SHVZK_real p).
```

Proof. See Coq-code. □

Theorem

Having established the preceding lemmas, we can now bound the advantage of the `Hiding`-games, denoted Adv_Hiding , to the sum of the advantages with which the adversary can distinguish between the corresponding `SHVZK`-games linked to each reduction module. We define this sum as our ε function and use the notation $\approx_\varepsilon$ to represent the distinguishing advantage in the `SHVZK`-games. Since the `SHVZK`-games, by definition of the Σ -protocol, are perfectly indistinguishable, we conclude that the commitment scheme satisfies *perfect hiding*. The formal statement of this result is given in the Coq code below; note that the `NoFail` hypotheses must be included.

```

Theorem Com_hiding_SHVZK :
  ∀ (p : raw_sigExt) ,
  (forall h w a s e, NoFail (response p h w a s e)) ->
  NoFail (sampl_challenge p) ->
  Adv_Hiding (sig_to_com p) (λ A,
    AdvFor (SHVZK p) (A ◦ Call_SHVZK_inp p) +
    AdvFor (SHVZK p) (A ◦ Call_SHVZK_sam p)).
    
```

Proof. Informally, fig. 13 shows the game transformation from `Hiding_real` to `Hiding_ideal` by either the lemmas or `AdvFor SHVZK`, exploiting the game-hopping, hybrid technique as described by Rosulek [13]. For a formal proof see `Coq` code. $\square$

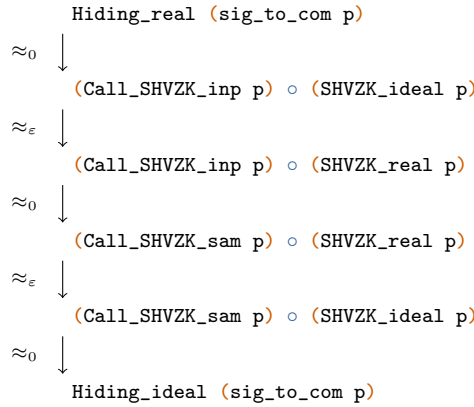


Figure 13: Hiding-game transformation

5.3 Binding

The *binding* property of the commitment scheme is defined in section 2.1 and modelled as the `Binding`-games in fig. 6. The games' `BINDING` operation take as input a commitment, c , and two pairs of values and openings, (v, o) and (v', o') . If the adversary succeeds in computing an input such that both pairs can open c , and $v \neq v'$, the `Binding_real` game returns `true`, otherwise `false`. The `Binding_ideal` game always returns `false`. The adversary can initialize the key used in the scheme through the `INIT` operation and retrieve the key through the `GET` operation. As in section 5.1 and section 5.2, we abstractly bound the advantage of the `Binding`-games, `Adv_Binding`, to some function, ε .

```

Definition Adv_Binding p (ε : adversary (IBinding p) → Axioms.R) :=
  ∀ A : adversary (IBinding p), AdvFor (Binding p) A <= ε A.
    
```

Proof strategy and lemmas

We want to show the close connection to the *special soundness* property of the underlying Σ -protocol (see fig. 14), and eventually that the advantage of the `Binding`-games, can be reduced to the *hardness* of the relation, R , (see fig. 15) on which the Σ -protocol is based.


```

Definition Special_Soundness_t p :
  game (Soundness p) :=
  [module no_locs ;
    #def #[ SOUNDNESS ] ('((h, a), ((e, z), (e', z')))) : 'soundness p) : 'bool
    ↪ {
      let b := p.(verify) h a e z in
      let b' := p.(verify) h a e' z' in
      let b'' := (e != e') in
      @ret 'bool (b && b' && b'')
    }
  ].

Definition Special_Soundness_f p :
  game (Soundness p) :=
  [module no_locs ;
    #def #[ SOUNDNESS ] ('((h, a), ((e, z), (e', z')))) : 'soundness p) : 'bool
    ↪ {
      let b := p.(verify) h a e z in
      let b' := p.(verify) h a e' z' in
      let b'' := (e != e') in
      let ow := p.(extractor) h a e e' z z' in
      @ret 'bool (if ow is Some w then ((p.(R) h w) && b && b' && b'') else
        ↪ false)
    }
  ].

```

Figure 14: Soundness-games as defined in `Sigma.v` of Nominal-SSProve

```

Definition Hardness (p: raw_sigExt) b :
  game (IHardness p) :=
  [module fset [:: key_loc (sig_to_com p) ] ;
    #def #[ INIT ] (_ : 'unit) : ('unit)
    {
      h ← (sig_to_com p).(setup) ;;
      #put key_loc (sig_to_com p) := Some h ;;
      ret tt
    } ;
    #def #[ GET ] (_ : 'unit) : ('key (sig_to_com p))
    {
      h ← getSome (key_loc (sig_to_com p));;
      ret h
    } ;
    #def #[ QUERY ] (w : 'witness p) : 'bool
    {
      h ← getSome (key_loc (sig_to_com p));;
      @ret 'bool (b && (p.(R) h w))
    }
  ].

```

Figure 15: Hardness-games differentiated by the boolean parameter `b`

The first perfect indistinguishability we want to show is between `Binding_real` and `Special_Soundness_t`:

$$\text{Binding_real } (\text{sig_to_com } p) \approx_0 \text{Special_Soundness_t } p$$

Recall that in fig. 3 we define `Commitment (c)` as `Message (a)`, `Value (v)` as `Challenge (e)`, `Opening (o)` as `Response (z)` and `Key (k)` as `Statement (h)`. Note also the structural similarity between the inputs to the `Binding` and `Soundness`-games; the only distinction is that the `Soundness`-games additionally take a `Statement`, `h`, as input. We construct the reduction module `Call_Soundness` in fig. 10 to retrieve the key from the location `key_loc` and forward it, along with the original input to `Binding_real`, to the `Special_Soundness_t` game. We formalise the perfect indistinguishability as the lemma below:

```
Lemma Binding_real_Soundness_t_perf p :
  perfect (IBinding (sig_to_com p))
    (Binding_real (sig_to_com p)) (Call_Soundness p ◦ Special_Soundness_t p).
```

Proof. Informally a code inspection of the two games reveals the equality of the two games. For a formal proof see `coq-code`. $\square$

Next we want to show perfect indistinguishability between the `Special_Soundness_f` game and the `Hardness true` game:

$$\text{Special_Soundness_f } p \approx_0 \text{Hardness } p \text{ true}$$

Since the `adversary` is defined using the binding interface `IBinding`, we need to compose both games with their corresponding reduction modules, i.e. `Call_Soundness` and `Call_Hardness` respectively of fig. 10 and 11, in order to compare them. The `Special_Soundness_f` game models the fact, that if an adversary can produce two accepting conversations (a, e, z) and (a, e', z') , then the Witness, w , can be computed efficiently using the `extractor` algorithm in fig. 2. The reduction `Call_Hardness` mimics this behavior, and forwards the final check $(p.(R) \ h \ w)$ to the `QUERY` operation of the `Hardness b` game. We formalise it as the lemma below:

```
Lemma Soundness_f_Hardness_true_perf p :
  perfect (IBinding (sig_to_com p))
    (Call_Soundness p ◦ Special_Soundness_f p) (Call_Hardness p ◦ Hardness p true).
```

Proof. See `coq-code`. $\square$

Lastly, we show perfect indistinguishability between `Hardness false` and `Binding_ideal`:

$$\text{Hardness } p \text{ false} \approx_0 \text{Binding_ideal } (\text{sig_to_com } p)$$

This follows naturally since both games always returns `false`.

```
Lemma Binding_ideal_Hardness_false_perf p :
  perfect (IBinding (sig_to_com p))
    (Call_Hardness p ◦ Hardness p false) (Binding_ideal (sig_to_com p)).
```

Proof. See `coq-code`. $\square$

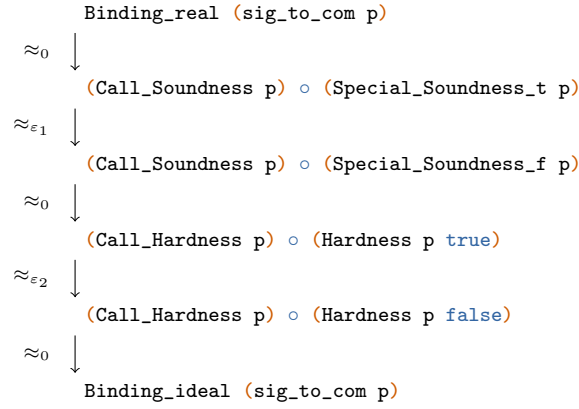


Figure 16: Binding-Game transformation

Theorem

Having established the preceding three lemmas, we now reduce the advantage of the **Binding**-games to the combined advantages of the **Soundness**- and **Hardness**-games. This reduction is expressed via our ε -function in `Adv_Binding`. We denote the advantage of the **Soundness**-games by $\approx_{\varepsilon_1}$ and that of the **Hardness** games by $\approx_{\varepsilon_2}$. According to definition 2, the **Soundness**-games are perfectly indistinguishable, yielding $\varepsilon_1 = 0$. Thus, the overall binding advantage reduces to that of the **Hardness**-games.

Consequently, we establish the *computational binding* property of the commitment scheme, as anticipated in section 4.1, since the adversary's probability of producing a valid `witness` that satisfies the relation R is negligible, for a hard relation. The formal statement of this result is provided in the `coq` code below:

```

Theorem Com_Binding_Soundness_Rel :
  ∀ (p : raw_sigExt) ,
  Adv_Binding (sig_to_com p) (λ A,
    AdvFor (Special_Soundness p) (A ∘ Call_Soundness p) +
    AdvFor (Hardness p) (A ∘ Call_Hardness p)).
    
```

Proof. Informally, fig. 16 shows the game transformation from `Binding_real` to `Binding_ideal` by either the lemmas, `AdvFor Special_Soundness` Or `AdvFor Hardness`. For a formal proof see `coq`-code. $\square$

6 Commitment scheme from Schnorr

Leveraging the modularity of state-separating proofs and the abstract, parameterized definitions and proofs we have shown above, we can now construct a commitment scheme from any Σ -protocol. As an example we take the Schnorr Σ -protocol as implemented in Nominal-SSProve³. The implementation instantiates the `raw_sigma` record from fig. 2, and it formally proves the SHVZK and Special Soundness properties of the Σ -protocol.

Given this implementation we can create a commitment scheme, `raw_comSchnorr`, using the function `sig_to_com` in fig. 3:

```

Definition raw_schnorrExt : raw_sigExt :=
  { | p := raw_schnorr

    ; sampl_challenge :=
      { code
        e ← sample uniform #|exp| ;;
        ret e
      }

    ; key_gen :=
      { code
        'w ← sample uniform #|exp| ;;
        ret (w, (fto (g ^+ otf w)))
      }
  }.

Definition raw_comSchnorr : raw_com := sig_to_com raw_schnorrExt.

```

Finally we use the `raw_comSchnorr` to populate the theorems of correctness, perfect hiding and computational binding below, and proof them using the properties of the already verified Schnorr protocol. For formal proofs we refer to the `ComSchnorr.v` file⁴.

```

Theorem comSchnorr_Correct : Adv_Correct raw_comSchnorr (λ _, 0).

Theorem comSchnorr_Hiding : Adv_Hiding raw_comSchnorr (λ _, 0).

Theorem comSchnorr_Binding : Adv_Binding raw_comSchnorr (λ A, AdvFor (Hardness
  ↪ raw_schnorrExt) (A ∘ Call_Hardness raw_schnorrExt)).

```

7 Conclusion

We have defined the semantics and properties of commitment schemes independent from other cryptographic primitives using the Nominal-SSProve framework. Furthermore we have fully formalised the transformation from Σ -protocol to commitment scheme as proposed by Damgård [7], resulting in a *perfectly hiding* commitment scheme with *computational binding*. Our work has been carried out in a modular fashion, that allow for any verified Σ -protocol to be transformed to a commitment scheme, exemplified with the Schnorr protocol.

³<https://github.com/MarkusKL/nominal-ssprove/blob/master/theories/Example/Sigma/Schnorr.v>

⁴<https://github.com/petersabroe/thesis/blob/main/project/theories/ComSchnorr.v>

References

- [1] Gilles Barthe, Daniel Hedin, Santiago Zanella Béguelin, Benjamin Grégoire, and Sylvain Heraud. A machine-checked formalization of sigma-protocols. In *Proceedings of the 2010 23rd IEEE Computer Security Foundations Symposium*, CSF '10, page 246–260, USA, 2010. IEEE Computer Society. doi:10.1109/CSF.2010.24.
- [2] Manuel Blum. Coin flipping by telephone. In *Advances in Cryptology: A Report on CRYPTO 81*, pages 11–15, 1981. URL: [/archive/crypto81/11_blum.pdf](#).
- [3] Chris Brzuska, Antoine Delignat-Lavaud, Cédric Fournet, Konrad Kohbrok, and Markulf Kohlweiss. State separation for code-based game-playing proofs. In *Advances in Cryptology – ASIACRYPT 2018: 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2–6, 2018, Proceedings, Part III*, page 222–249, Berlin, Heidelberg, 2018. Springer-Verlag. doi:10.1007/978-3-030-03332-3_9.
- [4] David Butler, Andreas Lochbihler, David Aspinall, and Adria Gascon. Formalising σ -protocols and commitment schemes using CryptHOL. Cryptology ePrint Archive, Paper 2019/1185, 2019. URL: <https://eprint.iacr.org/2019/1185>.
- [5] Ronald Cramer. *Modular Design of Secure yet Practical Cryptographic Protocols*. PhD thesis, CWI - Dutch National Research Institute for Mathematics and Computer Science, January 1997.
- [6] Ivan Bjerre Damgård. On the existence of bit commitment schemes and zero-knowledge proofs. In Gilles Brassard, editor, *Advances in Cryptology — CRYPTO' 89 Proceedings*, pages 17–27, New York, NY, 1990. Springer New York.
- [7] Ivan Damgård. On Σ -protocols. Lecture Notes, University of Aarhus, Department for Computer Science, 2010. v.2. URL: <https://www.cs.au.dk/~ivan/Sigma.pdf>.
- [8] Yevgeniy Dodis. Lecture 14 (notes). Course: Introduction to Cryptography, NYU. URL: https://piiazza.com/class_profile/get_resource/iiayue4zvjj2hu/inhhhsyt9x74kl.
- [9] Shimon Even. Protocol for signing contracts. In *Advances in Cryptology: A Report on CRYPTO 81*, pages 148–153, 1981. URL: [/archive/crypto81/148_even.pdf](#).
- [10] Philipp G. Haselwarter, Exequiel Rivas, Antoine Van Muylder, Théo Winterhalter, Carmine Abate, Nikolaj Sidorenko, Cătălin Hrițcu, Kenji Mailard, and Bas Spitters. Ssprove: A foundational framework for modular cryptographic proofs in coq. *ACM Trans. Program. Lang. Syst.*, 45(3), July 2023. doi:10.1145/3594735.

- [11] Markus Krabbe Larsen and Carsten Schürmann. Nominal state-separating proofs. Cryptology ePrint Archive, Paper 2025/598, 2025. URL: <https://eprint.iacr.org/2025/598>.
- [12] Roberto Metere and Changyu Dong. *Automated Cryptographic Analysis of the Pedersen Commitment Scheme*, page 275–287. Springer International Publishing, 2017. URL: http://dx.doi.org/10.1007/978-3-319-65127-9_22, doi:10.1007/978-3-319-65127-9_22.
- [13] Mike Rosulek. The joy of cryptography. <https://joyofcryptography.com>. URL: <https://joyofcryptography.com>.
- [14] Berry Schoenmakers. Lecture notes cryptographic protocols. Lecture Notes, Technical University of Eindhoven, Department of Mathematics and Computer Science, 2025. v.1.10. URL: <https://berry.win.tue.nl/CryptographicProtocols/LectureNotes.pdf>.